Northlake Implementation Guide

How Northlake University is built in EEMSuite, one stage at a time

PREPARED BY	Energy Hippo implementation team
STATUS	Stage 1 ready to enter; stages 2 to 10 authored; stage 11 research

Northlake University is the synthetic customer that has to exercise every EEMSuite capability: bills, providers, allocation, generation, custom templates, every gateway, tasks, ENERGY STAR, every report and some custom ones, views, a custom task. It is presented as a real customer so it can be translated into EEM the way any customer is, and it is built once, through the applications, into a database that becomes the reference restore for testing and demonstrations. When a capability is added to EEM, Northlake gets the data that exercises it.

This folder is that build. Everything in it is arranged so that the person entering data always knows what to open next.

How the folder is arranged

```
implementation/
  northlake-implementation-guide.pdf   this document
  vision.pdf                           why the synthetic customer exists
  roadmap.pdf                          what each stage unlocks, and the coverage scorecard
  report-coverage-matrix.pdf           every report, the data it needs, the stage that provides it
  product-findings.pdf                 things EEM supports but no screen can create; product work
  briefs/                              design briefs for the brand, theme, animations, intake package
  stage-1-foundation/                  one folder per stage:
    stage-1-foundation.pdf               the chapter: app, screen, file, record, check
    01-companies.yaml                    one YAML file per editor, numbered in entry order
    02-utility-providers.yaml
    ...
    checks.sql                           read-only queries that prove the stage is complete
  stage-2-hierarchy/
  ...
  stage-10-baseline/
  stage-11-third-party/              research so far: what needs a service outside the database
  stage-12-scenarios/                what happens once it is running: replacements, unit changes, rou
  te fixes, late reads
```

The customer's own documents stay in `documents/`: the intake package, guides, tariff sheets, bills and brand kit that Northlake "sent". The source model that generates those documents stays in `data/`. Nothing in `implementation/` is generated from a run; it is the record of the one build.

How to use a stage

1. Open the stage's chapter PDF. Read "Before you start" and confirm the previous stage's checks are green.
2. Work the steps in order. Each step names the application and the screen, links the YAML file that holds the values for that screen, and says in what order the editor wants its pages filled.
3. Keep the YAML file open beside the editor. Its section keys are the editor's page names and its field keys are the editor's labels, so the file reads the way the screen does. Type what it says.
4. After each save, write the generated ID into the record's `installed:` block. Later stages refer to records by their `key`, and the IDs are how a reader of the finished database traces a row back to its source.
5. When the last step is done, run the stage's `checks.sql` against the instance database. Each result set states what it expects in its heading.
6. If a screen cannot take a value the file asks for, do not fill the gap with SQL. Record it under the chapter's Findings and in product findings. A gap between what the product supports and what its screens can create is part of what Northlake exists to find.

The stages

Stage	Chapter	What it builds
1	Foundation	Companies with levels and defaults, utility providers with bill cycles and vendor numbers, fiscal calendars, groups with permissions, users
2	Hierarchy	Northlake's own units, billing accounts, then the whole estate as nodes: sites, buildings, meters, points, weather stations with degree days, station assignments, aggregates, baseline point shells
3	Gateways and data	Every gateway format, its workflow and task, the AcquiSuite acceptance, handheld routes and baselines
4	Rates	Schedules, holidays, seasons, time of use, every determinant kind, rate assignments and rate tests
5	Bills	Bill entry and import with charge lines, bill images, validation rules, status history, notes
6	Financial	GL accounts and fields, AP and GL export formats, approval and upload, accruals
7	Allocation and tenants	The Town Center company, every allocation method, weighted use, bill generation, reconciliation
8	Sustainability	Emission factors and GHG, ENERGY STAR, weather regression, benchmarks
9	Operations	Projects, alarms, audit, saved reports and dashboards, delivery to 100+ users, custom reports and views, a custom task, ongoing handheld rounds using Stage 3 configuration, importers, EnergyAI
10	Baseline	The backup, a restore into a clean instance, and the verification that makes it the reference database
11	Third-party integrations	Research only for now: identity (OIDC, SAML), SCIM provisioning, the RabbitMQ broker, mail, ENERGY STAR and EnergyAI accounts, the public API, and what a second server needs beside the restored database
12	Scenarios	Changes made while the system runs: a replaced handheld meter, a register that moves from kW to MW, a badly keyed route fixed with bulk edit, and a tenant's demand read the day after its energy

The roadmap says which reports each stage lights up and keeps the coverage scorecard. The report coverage matrix lists every report against the stage that feeds it.

Rules that hold in every stage

- **Through the applications, or through a real data path.** Database Administrator, Configuration Tool, Bill Importer, the web site, a gateway reading a file, an importer, an external service. Never a SQL insert of customer data. Installing the product's own SQL objects is standing up EEMSuite, not seeding a customer.
- **A missing screen is a finding.** Where the product supports something no screen can create, record it and move on. The product findings document is the backlog that results.
- **Covered means a known value renders.** A capability counts only when a real report or output shows Northlake data with a value that was expected beforehand. The stage checks and the report matrix carry those values.
- **The YAML is the implementation.** When a value has to change, change the YAML first and then the application, so the folder always matches the database. The customer documents in `documents/` are not edited to match; a real customer's packet drifts from the implementation too.
- **One build.** Northlake is entered once. The result of Stage 10 is the database that gets restored from then on. Stage 11 is what runs beside that database: the services and per-server settings the backup cannot carry. If EEM changes so much that the build must be repeated, the YAML is what makes that possible.

Conventions inside a YAML file

```
stage: 1
step: 2
title: Utility providers
app: Database Administrator
screen: Utility Providers
source: [data/providers/valley-electric-district.yaml]
notes:
  - Create the provider on Definitions and save before the other pages unlock.
records:
  - key: valley_electric_district
    installed: {providerId: 1}
    Definitions: {Name: Valley Electric District, Website: https://www.valleyelectric.example, Icon: f
  a-bolt-lightning}
    Remit Addresses:
      - {Address: PO Box 41027, City: Sacramento, Region: CA, Postal Code: "95841"}
```

`key` is stable and is how other files refer to the record. `installed` holds every ID EEM generated for it. Section names are the editor's own page names. Anything the product creates on its own (a register point's usage point, the degree-day points behind a temperature index, the default bill scenario) is described under `notes` and never listed as something to enter. Values marked `# assumed` were invented where the customer documents were silent.